

```
docker run -d -p 8080:8080 --name airflow apache/airflow standalone
```

```
docker ps
```

```
docker logs airflow
```

```
docker exec -it airflow bash
```

```
cd /opt/airflow/dags
```

```
cat > employee_salary_processing_dag.py
```

```
from airflow import DAG
```

```
from airflow.operators.python import PythonOperator
```

```
from datetime import datetime
```

```
EMPLOYEE_FILE = "/tmp/employees.txt"
```

```
REPORT_FILE = "/tmp/salary_report.txt"
```

```
def create_employee_file():
```

```
    employees = [
```

```
        "Rahul,45000",
```

```
        "Sneha,52000",
```

```
        "Amit,61000",
```

```
        "Priya,47000",
```

```
        "Kiran,39000"
```

```
    ]
```

```
    with open(EMPLOYEE_FILE, "w") as file:
```

```
        for employee in employees:
```

```
            file.write(employee + "\n")
```

```
    print("Employee file created successfully")
```

```
def read_employee_data():
```

```
    with open(EMPLOYEE_FILE, "r") as file:
```

```

    data = file.readlines()

    print("Employee Records:")

    for line in data:

        print(line.strip())

def calculate_salary_expense():

    total_salary = 0

    with open(EMPLOYEE_FILE, "r") as file:

        for line in file:

            name, salary = line.strip().split(",")

            total_salary += int(salary)

    print(f"Total Salary Expense = {total_salary}")

def find_highest_salary():

    with open(EMPLOYEE_FILE, "r") as file:

        data = file.readlines()

        highest_salary = 0

        employee_name = ""

        for line in data:

            name, salary = line.strip().split(",")

            salary = int(salary)

            if salary > highest_salary:

                highest_salary = salary

                employee_name = name

        print(f"Highest Salary = {highest_salary}")

        print(f"Employee = {employee_name}")

def generate_salary_report():

    total_salary = 0

    employee_count = 0

    with open(EMPLOYEE_FILE, "r") as file:

```

```

    for line in file:

        name, salary = line.strip().split(",")

        total_salary += int(salary)

        employee_count += 1

    report = f"""
Employee Salary Report

Total Employees = {employee_count}

Total Salary Expense = {total_salary}

Status = Processed Successfully

"""

    with open(REPORT_FILE, "w") as file:

        file.write(report)

    print("Salary report generated successfully")

with DAG(
    dag_id="employee_salary_processing",
    start_date=datetime(2024, 1, 1),
    schedule=None,
    catchup=False
) as dag:

    create_employee_file_task = PythonOperator(
        task_id="create_employee_file",
        python_callable=create_employee_file
    )

    read_employee_data_task = PythonOperator(
        task_id="read_employee_data",
        python_callable=read_employee_data
    )

    calculate_salary_expense_task = PythonOperator(

```

```

        task_id="calculate_salary_expense",
        python_callable=calculate_salary_expense
    )
    find_highest_salary_task = PythonOperator(
        task_id="find_highest_salary",
        python_callable=find_highest_salary
    )
    generate_salary_report_task = PythonOperator(
        task_id="generate_salary_report",
        python_callable=generate_salary_report
    )

    create_employee_file_task >> read_employee_data_task >>
    calculate_salary_expense_task >> find_highest_salary_task >>
    generate_salary_report_task

```

```
airflow@1a750779e533:/opt/airflow/dags$ cat /tmp/employees.txt
```

Rahul,45000

Sneha,52000

Amit,61000

Priya,47000

Kiran,39000

```
airflow@1a750779e533:/opt/airflow/dags$ cat /tmp/salary_report.txt
```

Employee Salary Report

Total Employees = 5

Total Salary Expense = 244000

Status = Processed Successfully

docker restart airflow

Dag
employee_salary_processing

ag

00:00:05.251

00:00:02.62

create_employee_file

read_employee_data

calculate_salary_expense

find_highest_salary

generate_salary_report

✓

✓

✓

✓

✓

✗

✓

✓

✓

✓

Search DagsCtrl+K

Trigger

employee_salary_processing

Schedule

Latest Run
2026-05-09 12:05:07 ✓

Next Run

Max Active Runs
0 of 16

Owner
airflow

Tags

Latest Dag Version
v3

Overview

Runs

Tasks

Calendar

Audit Log

Code

Details

+ Filter

3 Dag Runs

Dag Run ID	Run After	State	Run Type	Triggering User	Start Date	Duration	Dag Version(s)	
manual_2026-05-09T06:35:07.952483+00:00	2026-05-09 12:05:07	✓ Success	▶ Manual	admin	2026-05-09 12:05:08	00:00:05.251	v3	C
manual_2026-05-09T06:34:28.259045+00:00	2026-05-09 12:04:28	✓ Success	▶ Manual	admin	2026-05-09 12:04:29	00:00:05.197	v3	C
manual_2026-05-09T06:31:06.730472+00:00	2026-05-09 12:01:06	✗ Failed	▶ Manual	admin	2026-05-09 12:01:07	00:00:04.511	v1	C